Reviewer Pack — Minimal Order of Braided Monoids and Resolution Proof Width

Entry 3999b93633dd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 08:13:45 UTC

Minimal Order of Braided Monoids and Resolution Proof Width

Entry ID: `3999b93633dd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 08:13:45 UTC

1. Conjecture

Field A (mathematical branch): Braids in Group Theory **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> For every boolean satisfiability problem φ, the resolution proof width w(φ) is linearly correlated with the minimal order of a braided monoid that can be generated by the variables and clauses of φ.

Rationale (proposer's reasoning):

> Braided monoids are an algebraic structure that has been used to model certain aspects of braid groups. If there exists a relationship between resolution proof width and the properties of braided monoids, it could provide new insights into the complexity of satisfiability problems.

Taxonomy category: `BraidsGroupTheory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3af8f43c1500771d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the linear regression coefficient (r^2) between minimal order of braided monoids and resolution proof width exceeds 0.9 for all seeds, with no seed having a correlation below 0.7.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Order Braided Monoids AND Resolution Proof Width` - `Boolean Satisfiability Resolution Proof Complexity related to Braids in Group Theory` - `Correlation between Minimal Order of Braided Monoids and Boolean Satisfiability Problem Resolution`

Top relevant hits considered: - [http://arxiv.org/abs/0712.3836v2] The well-ordering of dual braid monoids - [http://arxiv.org/abs/math/0512165v3] Classification of braids which give rise to interchange - [http://arxiv.org/abs/1601.00169v5] Braided quantum groups and their bosonizations in the C^*-algebraic framework - [http://arxiv.org/abs/2309.16111v2] The relational complexity of linear groups acting on subspaces - [http://arxiv.org/abs/1110.6618v2] Brauer relations in finite groups II - quasi-elementary groups of order p^aq - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/2206.00903v2] Satisfiability of Quantified Boolean Announcements - [http://arxiv.org/abs/0907.0930v3] Classifying spaces for braided monoidal categories and lax diagrams of bicategories

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n_vars, n_clauses):
        variables = list(range(1, n_vars + 1))
        clauses = []
        for _ in range(n_clauses):
            clause = [random.choice(variables), random.choice(variables)]
            while len(set(clause)) < 2:
                clause = [random.choice(variables), random.choice(variables)]
            clauses.append(tuple(sorted(clause)))
        return variables, set(clauses)

    def resolution_width(phi):
        n_vars = max(phi[0])
        n_clauses = len(phi[1])

        # Simplified resolution width calculation
        return n_clauses + n_vars

    def minimal_braided_monoid_order(phi):
        n_vars = max(phi[0])
        n_clauses = len(phi[1])
```

```
        # Simplified braided monoid order calculation
        return n_vars + n_clauses

    results = []
    for _ in range(30):
        n_vars = random.randint(5, 40)
        n_clauses = random.randint(n_vars, n_vars * 2)
        phi = generate_instance(n_vars, n_clauses)

        w_phi = resolution_width(phi)
        n_braided_monoid = minimal_braided_monoid_order(phi)

        results.append((n_braided_monoid, w_phi))

    if not results:
        return {
            "metric_name": "resolution_width",
            "metric_value": 0,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "No instances generated"
        }

    n_braided_monoid_values = [r[0] for r in results]
    w_phi_values = [r[1] for r in results]

    mean_n_braided_monoid = sum(n_braided_monoid_values) / len(n_braided_monoid_values)
    mean_w_phi = sum(w_phi_values) / len(w_phi_values)

    covariance = sum((n_braided_monoid_values[i] - mean_n_braided_monoid) * (w_phi_values[i] - mea
    variance_n_braided_monoid = sum((n_braided_monoid_values[i] - mean_n_braided_monoid) ** 2 for

    if variance_n_braided_monoid == 0:
        return {
            "metric_name": "resolution_width",
            "metric_value": 0,
            "instances_tested": len(results),
            "n_max": max(max(phi[0]) for phi in results),
            "conjecture_holds": False,
            "counterexample": "Variance of n_braided_monoid is zero"
        }

    r_squared = covariance / (math.sqrt(variance_n_braided_monoid) * math.sqrt(sum((w_phi_values[i

    return {
        "metric_name": "resolution_width",
        "metric_value": r_squared,
        "instances_tested": len(results),
        "n_max": max(max(phi[0]) for phi in results),
        "conjecture_holds": 0.7 <= r_squared < 0.9,
        "counterexample": ""
    }

if __name__ == "__main__":
```

```
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_r_squared = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_r_squared} std=0 support_fraction={support_fraction}"
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"r_squared_outside_bounds\" first_failing_seed={
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe99c73b.py", line 101, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe99c73b.py", line 91, in run_trial
    "n_max": max(max(phi[0]) for phi in results),
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe99c73b.py", line 91, in <genexpr>
    "n_max": max(max(phi[0]) for phi in results),
                 ^^^^^^^^^^
TypeError: 'int' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the required linear regression coefficient (r^2) could not be calculated to verify the conjecture. | next: Re-run the test with proper error handling and ensure it completes without crashing to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14062
2	preregistration	ollama_remote	glm4:latest	0	0	9363
3	novelty	ollama_remote	glm4:latest	0	0	8477
4	novelty	ollama_remote	glm4:latest	0	0	10194
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18824
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10959
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7819
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13122
9	judge	ollama_remote	glm4:latest	0	0	12659

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 105478 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3999b93633dd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3999b93633dd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3999b93633dd.tar.gz` (if generated)